

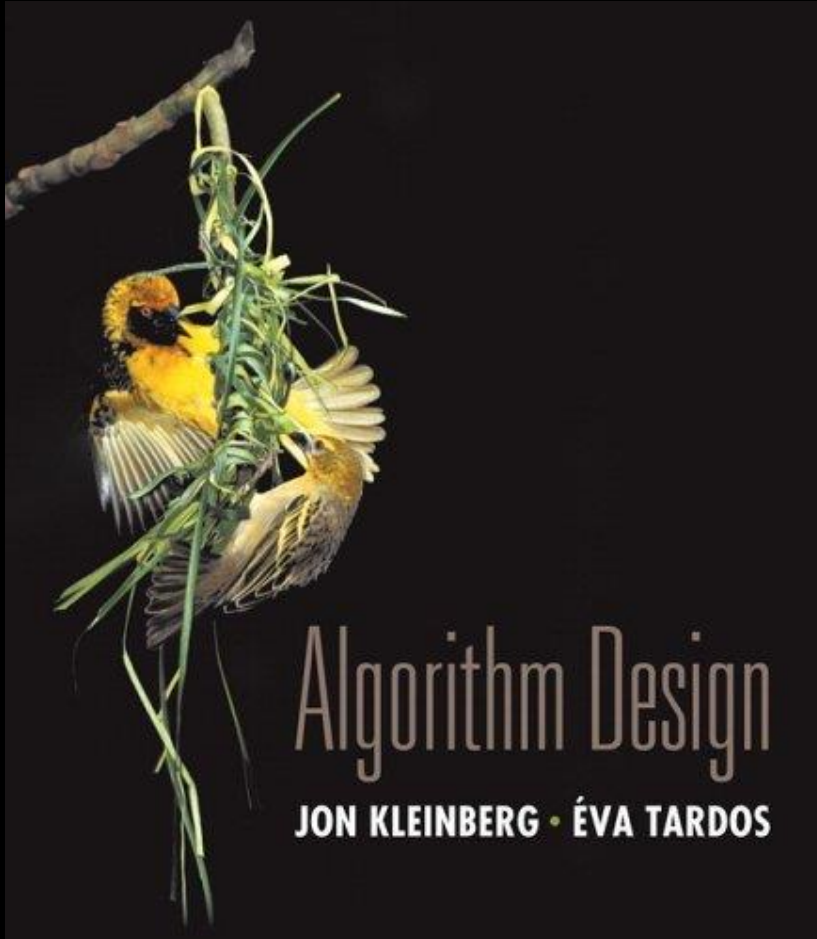
Course evaluation

<https://evaluation.shanghaitech.edu.cn/tqnmaep/login>



Chapter 11

Approximation Algorithms



Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

Approximation Algorithms

Q. Suppose I need to solve an NP-hard problem. What should I do?

A. Theory says you're unlikely to find a poly-time algorithm.

Must sacrifice one of three desired features.

- Solve problem to optimality.
- Solve problem in poly-time.
- Solve arbitrary instances of the problem.

ρ -approximation algorithm.

- Guaranteed to run in poly-time.
- Guaranteed to solve arbitrary instance of the problem
- Guaranteed to find solution within ratio ρ of true optimum.

Challenge. Need to prove a solution's value is close to optimum, without even knowing what optimum value is!

11.1 Load Balancing

Load Balancing

Input. m identical machines; n jobs, job j has processing time t_j .

- Job j must run contiguously on one machine.
- A machine can process at most one job at a time.

Def. Let $J(i)$ be the subset of jobs assigned to machine i . The **load** of machine i is $L_i = \sum_{j \in J(i)} t_j$.

Def. The **makespan** is the maximum load on any machine $L = \max_i L_i$.

Load balancing. Assign each job to a machine to minimize makespan.



Load Balancing

Input. m identical machines; n jobs, job j has processing time t_j .

- Job j must run contiguously on one machine.
- A machine can process at most one job at a time.

Def. Let $J(i)$ be the subset of jobs assigned to machine i . The **load** of machine i is $L_i = \sum_{j \in J(i)} t_j$.

Def. The **makespan** is the maximum load on any machine $L = \max_i L_i$.

Load balancing. Assign each job to a machine to minimize makespan.

Claim. Load balancing is NP-hard.

Pf. $\text{NUMBER-PARTITION} \leq_p \text{LOAD-BALANCE}$

Partition

SUBSET-SUM. Given natural numbers $w_1, \dots, w_n$ and an integer W , is there a subset that adds up to exactly W ?

PARTITION. Given natural numbers $v_1, \dots, v_m$, can they be partitioned into **two** subsets that add up to the same value?

$$\nwarrow \frac{1}{2} \sum_i v_i$$

Claim. SUBSET-SUM $\leq_p$ PARTITION.

Pf. Let $W, w_1, \dots, w_n$ be an instance of SUBSET-SUM.

- Create instance of PARTITION with $m = n+2$ elements.

$$- v_1 = w_1, v_2 = w_2, \dots, v_n = w_n, \quad v_{n+1} = 2 \sum_i w_i - W, \quad v_{n+2} = \sum_i w_i + W$$

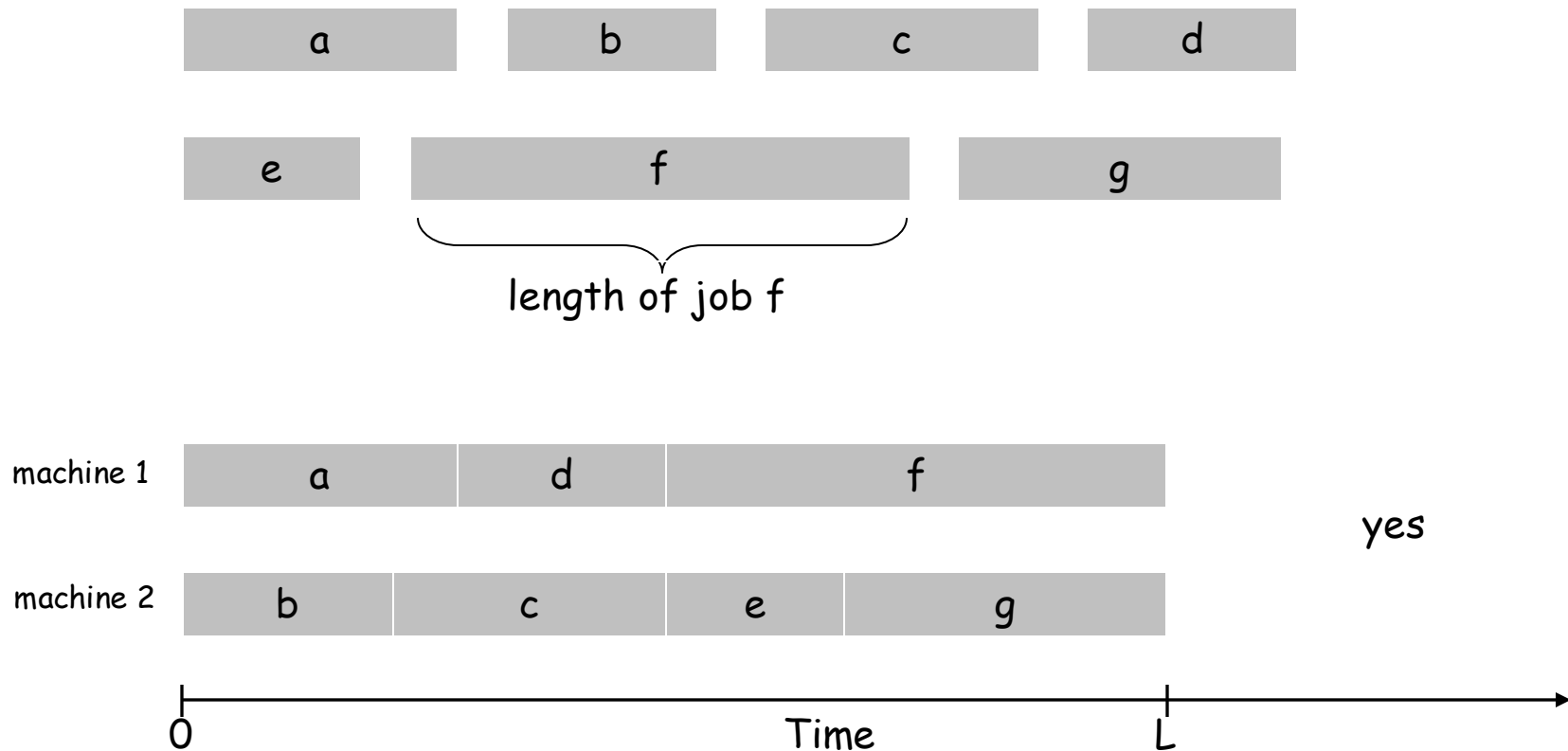
- There exists a subset that sums to W iff there exists a partition since two new elements cannot be in the same partition. ▀

$v_{n+1} = 2 \sum_i w_i - W$	W	subset A
$v_{n+2} = \sum_i w_i + W$	$\sum_i w_i - W$	subset B

Load Balancing on 2 Machines

Claim. Load balancing is NP-hard.

Pf. $\text{NUMBER-PARTITION} \leq_p \text{LOAD-BALANCE}$.



Load Balancing: List Scheduling

List-scheduling algorithm.

- Consider n jobs in some fixed order.
- Assign job j to machine whose load is smallest so far.

```
List-Scheduling( $m, n, t_1, t_2, \dots, t_n$ ) {  
  for  $i = 1$  to  $m$  {  
     $L_i \leftarrow 0$             $\leftarrow$  load on machine  $i$   
     $J(i) \leftarrow \phi$         $\leftarrow$  jobs assigned to machine  $i$   
  }  
  
  for  $j = 1$  to  $n$  {  
     $i = \operatorname{argmin}_k L_k$        $\leftarrow$  machine  $i$  has smallest load  
     $J(i) \leftarrow J(i) \cup \{j\}$   $\leftarrow$  assign job  $j$  to machine  $i$   
     $L_i \leftarrow L_i + t_j$      $\leftarrow$  update load of machine  $i$   
  }  
}
```

Implementation. $O(n \log n)$ using a priority queue.

Load Balancing: List Scheduling Analysis

Theorem. [Graham, 1966] Greedy algorithm is a 2-approximation.

- First worst-case analysis of an approximation algorithm.
- Need to compare resulting solution with optimal makespan L^* .

Lemma 1. The optimal makespan $L^* \geq \max_j t_j$.

Pf. Some machine must process the most time-consuming job. ▫

Lemma 2. The optimal makespan $L^* \geq \frac{1}{m} \sum_j t_j$.

Pf.

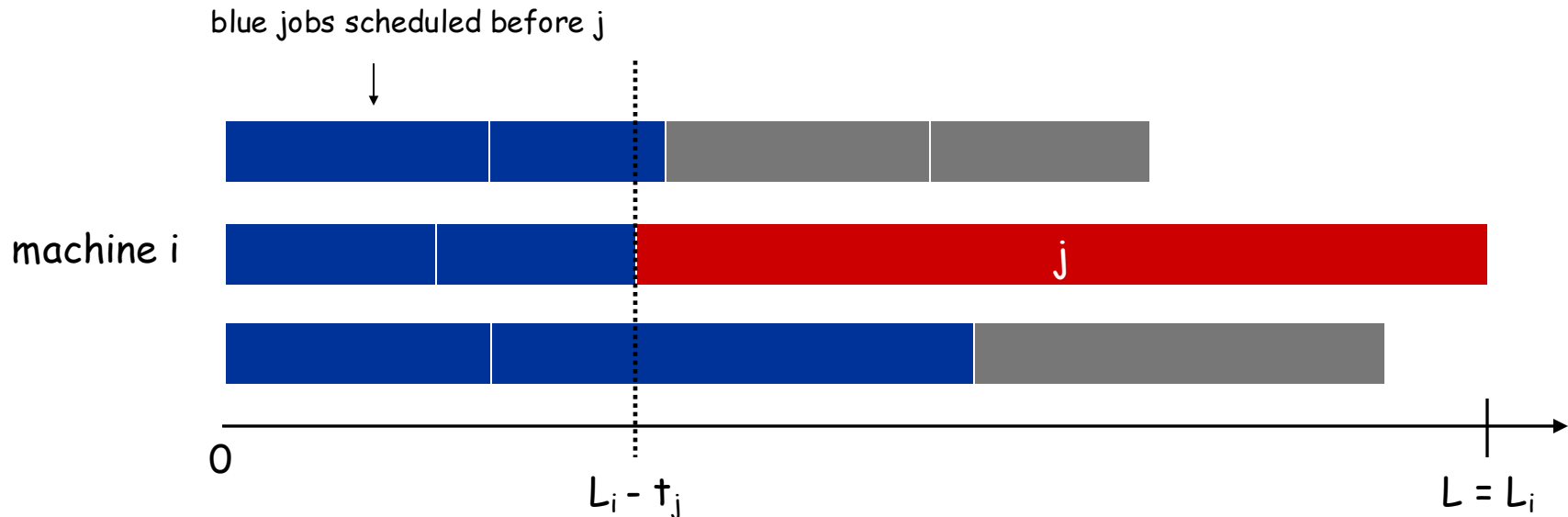
- The total processing time is $\sum_j t_j$.
- One of m machines must do at least a $1/m$ fraction of total work. ▫

Load Balancing: List Scheduling Analysis

Theorem. Greedy algorithm is a 2-approximation.

Pf. Consider load L_i of bottleneck machine i .

- Let j be last job scheduled on machine i .
- When job j assigned to machine i , i had smallest load:
 $L_i - t_j \leq L_k$ for all $1 \leq k \leq m$.



Load Balancing: List Scheduling Analysis

Theorem. Greedy algorithm is a 2-approximation.

Pf. Consider load L_i of bottleneck machine i .

- Let j be last job scheduled on machine i .
- When job j assigned to machine i , i had smallest load:
 $L_i - t_j \leq L_k$ for all $1 \leq k \leq m$.
- Sum inequalities over all k and divide by m :

$$\begin{aligned} L_i - t_j &\leq \frac{1}{m} \sum_{k=1}^m L_k \\ &= \frac{1}{m} \sum_{l=1}^n t_l \\ \text{Lemma 2} \rightarrow &\leq L^* \end{aligned}$$

$$\begin{aligned} \text{▫ Now } L_i &= \underbrace{(L_i - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq L^*} \leq 2L^*. \\ &\quad \uparrow \\ &\quad \text{Lemma 1} \end{aligned}$$

Load Balancing: List Scheduling Analysis

Q. Is our analysis tight?

A. Essentially yes.

Ex: m machines, $m(m-1)$ jobs length 1 jobs, one job of length m
 $L = 2m-1$;

$m = 10$

										machine 2 idle
										machine 3 idle
										machine 4 idle
										machine 5 idle
										machine 6 idle
										machine 7 idle
										machine 8 idle
										machine 9 idle
										machine 10 idle

list scheduling makespan = 19

Load Balancing: List Scheduling Analysis

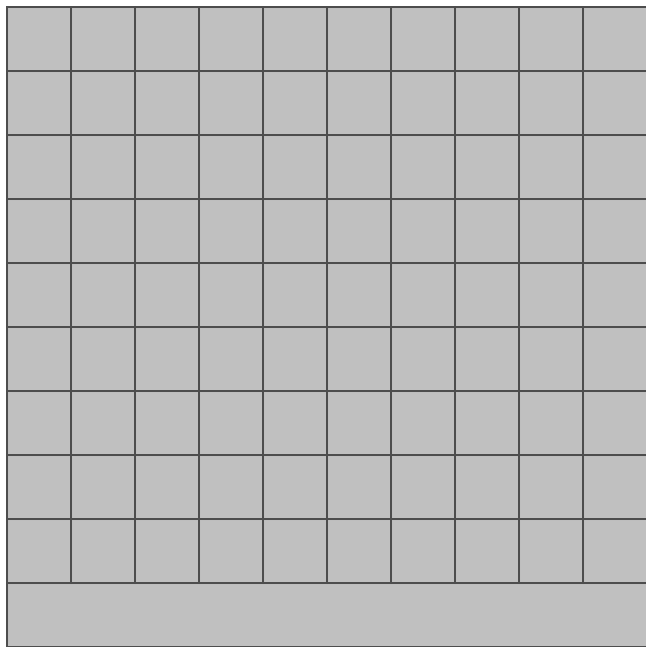
Q. Is our analysis tight?

A. Essentially yes.

Ex: m machines, $m(m-1)$ jobs length 1 jobs, one job of length m

$$L = 2m-1; L^* = m$$

$m = 10$



optimal makespan = 10

Load Balancing: LPT Rule

Longest processing time (LPT). Sort n jobs in descending order of processing time, and then run list scheduling algorithm.

```
LPT-List-Scheduling( $m, n, t_1, t_2, \dots, t_n$ ) {  
    Sort jobs so that  $t_1 \geq t_2 \geq \dots \geq t_n$   
  
    for  $i = 1$  to  $m$  {  
         $L_i \leftarrow 0$             $\leftarrow$  load on machine  $i$   
         $J(i) \leftarrow \phi$         $\leftarrow$  jobs assigned to machine  $i$   
    }  
  
    for  $j = 1$  to  $n$  {  
         $i = \operatorname{argmin}_k L_k$             $\leftarrow$  machine  $i$  has smallest load  
         $J(i) \leftarrow J(i) \cup \{j\}$     $\leftarrow$  assign job  $j$  to machine  $i$   
         $L_i \leftarrow L_i + t_j$         $\leftarrow$  update load of machine  $i$   
    }  
}
```

Load Balancing: LPT Rule

Observation. If at most m jobs, then list-scheduling is optimal.

Pf. Each job put on its own machine. ▀

Lemma 3. If there are more than m jobs, $L^* \geq 2 t_{m+1}$.

Pf.

- Consider first $m+1$ jobs $t_1, \dots, t_{m+1}$.
- Since the t_i 's are in descending order, each takes at least t_{m+1} time.
- There are $m+1$ jobs and m machines, so by pigeonhole principle, at least one machine gets two jobs. ▀

Theorem. LPT rule is a $3/2$ approximation algorithm.

Pf. If max-load machine M_i has only one job, then it's optimal (lemma 1)
Otherwise, for its last job t_j we have $j > m$. Use the same approach as for list scheduling.

$$L_i = \underbrace{(L_i - t_j)}_{\leq L^*} + \underbrace{t_j}_{\leq \frac{1}{2}L^*} \leq \frac{3}{2}L^*.$$

← Lemma 3

Load Balancing: LPT Rule

Q. Is our $3/2$ analysis tight?

A. No.

Theorem. [Graham, 1969] LPT rule is a $4/3$ -approximation.

Pf. More sophisticated analysis of same algorithm.

Q. Is Graham's $4/3$ analysis tight?

A. Essentially yes.

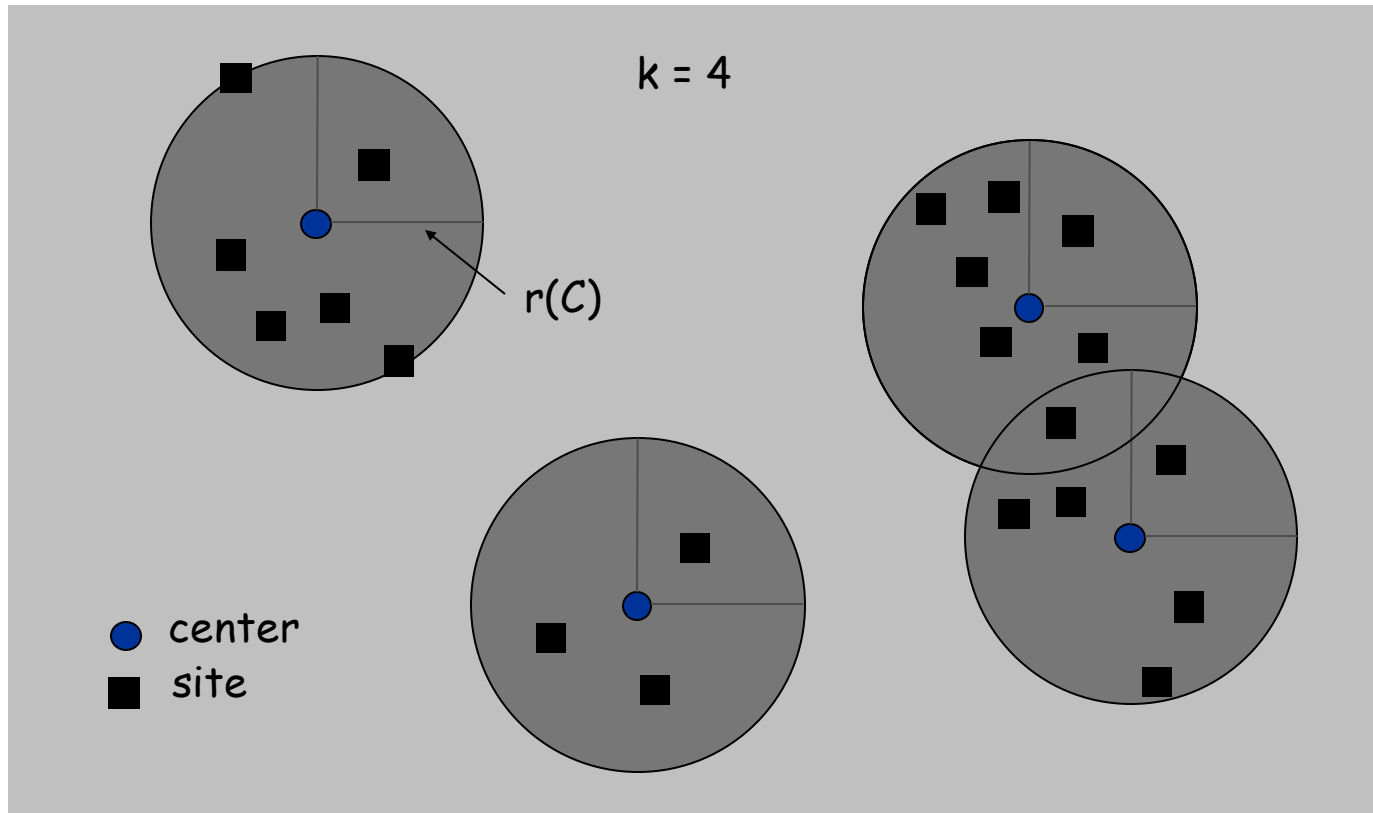
Ex: m machines, $n = 2m+1$ jobs, 2 jobs of length $2m-1, 2m-2, \dots, m+1$ and 3 jobs of length m .

11.2 Center Selection

Center Selection Problem

Input. Set of n sites $s_1, \dots, s_n$.

Center selection problem. Select k centers C so that maximum distance from a site to nearest center is minimized.



Center Selection Problem

Input. Set of n sites $s_1, \dots, s_n$.

Center selection problem. Select k centers C so that maximum distance from a site to nearest center is minimized.

Notation.

- $\text{dist}(x, y)$ = distance between x and y .
- $\text{dist}(s_i, C) = \min_{c \in C} \text{dist}(s_i, c)$ = distance from s_i to closest center.
- $r(C) = \max_i \text{dist}(s_i, C)$ = smallest covering radius.

Goal. Find set of centers C that minimizes $r(C)$, subject to $|C| = k$.

Distance function properties.

- $\text{dist}(x, x) = 0$ (identity)
- $\text{dist}(x, y) = \text{dist}(y, x)$ (symmetry)
- $\text{dist}(x, y) \leq \text{dist}(x, z) + \text{dist}(z, y)$ (triangle inequality)

Center Selection: Greedy Algorithm

Greedy algorithm. Repeatedly choose the next center to be the site **farthest** from any existing center.

```
Greedy-Center-Selection( $k, n, s_1, s_2, \dots, s_n$ ) {  
   $C = \{s\}$ ,  $s$  is any site  
  repeat  $k-1$  times {  
    Select a site  $s_i$  with maximum  $\text{dist}(s_i, C)$   
    Add  $s_i$  to  $C$   
  }  
  return  $C$   
}
```

↑
site farthest from any center

Lemma. Upon termination all centers in C are pairwise at least $r(C)$ apart.

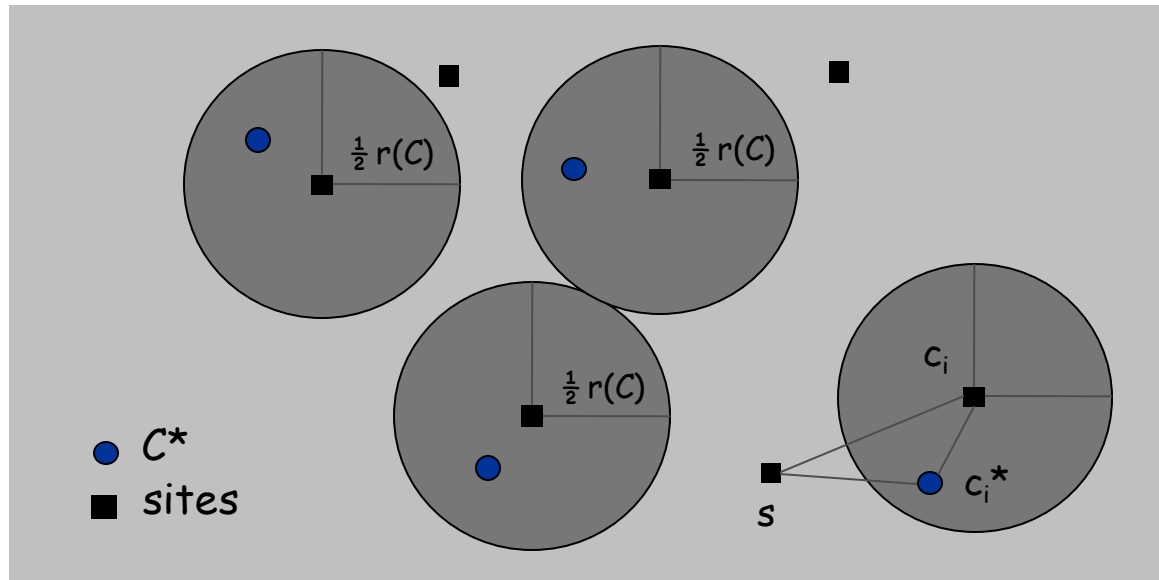
Pf. In the algorithm, the distance from a new center to other centers is at least the value of $r(C)$ before the new center is added; with more centers, $r(C)$ always monotonically decreases.

Center Selection: Analysis of Greedy Algorithm

Theorem. Let C^* be an optimal set of centers. Then $r(C) \leq 2r(C^*)$.

Pf. (by contradiction) Assume $r(C^*) < \frac{1}{2} r(C)$.

- For each center/site c_i in C , consider ball of radius $\frac{1}{2} r(C)$ around it.
 - No overlap between balls (by lemma)
- At least one $c_i^* \in C^*$ in each ball (because $r(C^*) < \frac{1}{2} r(C)$)
 - Therefore, exactly one $c_i^* \in C^*$ in each ball

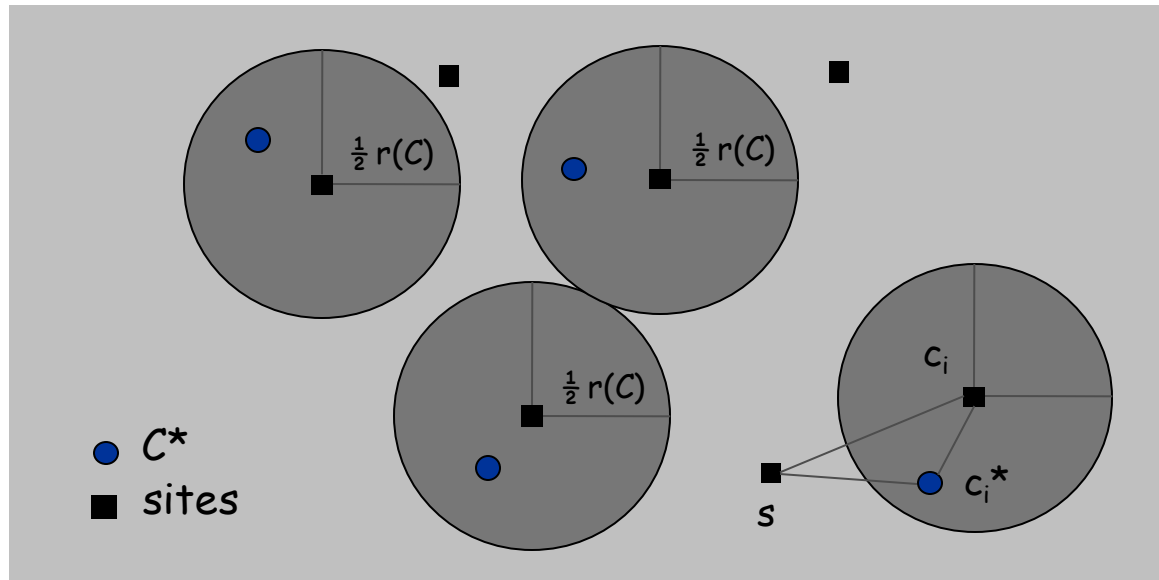


Center Selection: Analysis of Greedy Algorithm

Theorem. Let C^* be an optimal set of centers. Then $r(C) \leq 2r(C^*)$.

Pf. (by contradiction) Assume $r(C^*) < \frac{1}{2} r(C)$.

- Consider any site s and its closest center c_i^* in C^* .
- $\text{dist}(s, C) \leq \text{dist}(s, c_i) \leq \text{dist}(s, c_i^*) + \text{dist}(c_i^*, c_i) \leq 2r(C^*)$.
- Thus $r(C) \leq 2r(C^*)$.
 - Δ -inequality
 - $\leq r(C^*)$ since c_i^* is closest center



Center Selection

Theorem. Let C^* be an optimal set of centers. Then $r(C) \leq 2r(C^*)$.

Theorem. Greedy algorithm is a 2-approximation for center selection problem.

Remark. Greedy algorithm always places centers at sites, but is still within a factor of 2 of best solution that is allowed to place centers anywhere.

↖
e.g., points in the plane

Center Selection: Hardness of Approximation

Question. Is there hope of a $3/2$ -approximation? $4/3$?

Theorem. Unless $P = NP$, there is no ρ -approximation algorithm for metric k -center problem for any $\rho < 2$.

Proof idea. Show how we could use a $(2 - \epsilon)$ approximation algorithm for k -center to solve an NP-complete problem (DOMINATING-SET) in poly-time.